Intrinisically Typed Syntax That Works (Functional Pearl)

ANONYMOUS AUTHOR(S)

Intrinsically typed syntax promises elegant mechanizations: well-typed terms are represented by well-typed data, substitution preserves typing by construction, and type safety proofs collapse to straightforward induction. For simple calculi such as the simply-typed lambda calculus this promise is fulfilled. For realistic systems, however, the technique often becomes impractical. In dependently typed proof assistants, intrinsically typed encodings of polymorphic calculi lead to pervasive type-changing equalities: renamings and substitutions indexed by other substitutions force proofs to transport terms across equal types that are not definitionally equal. This phenomenon—informally known as transfer hell—causes even routine metatheoretic arguments to be dominated by administrative reasoning.

This functional pearl shows how to make intrinsically typed syntax work in practice for polymorphic calculi. We give extacts of a mechanization of the metatheory of System F and extend it to System Fω in Agda while avoiding transfer reasoning almost entirely. The key idea is to internalize the equational theory of the σ-calculus for substitutions as definitional equality. We rely on proved σ-calculus laws for a functional representation of substitutions and install them into Agda's rewrite mechanism, obtaining canonical behavior for substitution and renaming without explicit transports. The main technical challenge is ensuring that the resulting rewrite system remains terminating and confluent together with Agda's built-in definitional equality.

The resulting development dramatically simplifies proofs: typing preservation becomes definitional, substitution lemmas disappear, and type conversion in System Fω can be handled intrinsically. Our experience suggests a general methodology: when intrinsically typed syntax generates transports, the correct solution is often not more lemmas but better definitional equality. We distill practical guidelines for mechanizing typed calculi in dependently typed proof assistants.

ACM Reference Format:
Anonymous Author(s). 2018. Intrinisically Typed Syntax That Works (Functional Pearl). *J. ACM* 37, 4, Article 111 (August 2018), 19 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Intrinsically typed syntax is widely recommended for mechanizing type systems in dependently typed proof assistants. By representing well-typed terms as well-typed data, typing invariants hold by construction as in a Church-style presentation, i.e., making typing preservation trivial. For small calculi this approach works beautifully. However, for realistic polymorphic calculi such as System F, mechanizations often become complicated: conceptually simple proofs become cluttered with transports across equal types and administrative equalities.

The simply-typed lambda calculus is the prime example where intrinsically typed syntax works like a charm. Figure 1 shows typical definitions of the syntax of types, type contexts, variables, and expressions, which are indexed by contexts and types (left column). Building on these definitions it is straightforward to define a small-step operational semantics and proving type safety. This is because subject reduction holds by construction and progress follows by a simple inductive proof (right column). The style of these definitions is adapted from the textbook by Kokke et al. [7].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-735X/2018/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

```
data Type : Set where
  𝟙     : Type
  _⇒_ : Type → Type → Type

data Ctx : Set where
  ∅     : Ctx
  _▷_   : Ctx → Type → Ctx

data _∋_ : Ctx → Type → Set where
  here  : (Γ ▷ T) ∋ T
  there : Γ ∋ T → (Γ ▷ U) ∋ T

data _⊢_ Γ : Type → Set where
  con : Γ ⊢ 𝟙
  var : Γ ∋ T → Γ ⊢ T
  lam : (Γ ▷ T) ⊢ U → Γ ⊢ (T ⇒ U)
  app : Γ ⊢ (T ⇒ U) → Γ ⊢ T → Γ ⊢ U

data _⟶_ {Γ} {T} : Γ ⊢ T → Γ ⊢ T → Set where
  β-lam : app {Γ = Γ} (lam e₁) e₂ ⟶ (e₁ [ e₂ ])
  ξ-app : e₁ ⟶ e₁′ → app e₁ e₂ ⟶ app e₁′ e₂

data Value {Γ} : Γ ⊢ T → Set where
  con : Value con
  lam : (e : (Γ ▷ T) ⊢ U) → Value (lam e)

data Progress : Γ ⊢ T → Set where
  done : Value e → Progress e
  step : e ⟶ e′ → Progress e

progress : (e : ∅ ⊢ T) → Progress e
progress con       = done con
progress (lam e) = done (lam e)
progress (app e₁ e₂)
  with progress e₁
... | step x       = step (ξ-app x)
... | done (lam e) = step β-lam
```

Fig. 1. Call-by-name simply-typed lambda calculus

We elide the definition of the substitution operator _[_], as we discuss substitutions in detail in Section 2.1.

The simply-typed lambda calculus therefore represents the ideal scenario: intrinsic typing elides type preservation proofs, thus making type safety proofs so concise that they can serve as textbook examples [7]. Nothing comparable to transport reasoning arises in this development as only definitional properties of substitutions are required. The remainder of this paper explains why the same approach breaks down for polymorphic calculi and how to recover some of its simplicity.

Transfer hell is the colloquial term for being forced to apply type-changing lemmas to terms that are already known to be equal. The root problem is that substitutions that are propositionally equal are not definitionally equal, so terms must repeatedly be transported across equal types. This situation is annoying because proof assistants like Agda or Coq plainly refuse to let users even state this equality.[1] This phenomenon appears when substitutions themselves carry dependent typing information. In System F and Fω, expression substitutions depend on type substitutions and type substitutions depend on renamings; composing them yields types that are equal only propositionally, forcing transports throughout proofs. One workaround is to state definitions and proofs with explicit equivalence relations, but this practice is cumbersome, inefficient, and far away from the holy grail of definitional equality, which just works.

To our knowledge, previous intrinsically typed mechanization of System Fω (e.g., [3]) follow a natural design: expression syntax is indexed by typing derivations and substitutions are indexed by substitutions at the type level. While elegant at the level of definitions, this design makes proofs brittle. Because expression substitutions depend on type substitutions, and type substitutions depend on renamings, which again depend on type renamings, routine equalities require repeated transport across equal types, leading to directly to transfer hell.

[1] A situation that can be addressed using heterogeneous equality introduced in Conor McBride's thesis [8]. Benton et al. [2] discuss this approach in the context of an adequacy proof for a logical relation.

The message of this pearl is simple: when intrinsically typed syntax becomes complicated, the problem is often not the encoding of terms but the definition of equality on substitutions. Strengthening definitional equality can simplify metatheory more effectively than adding automation or additional lemmas.

In this paper, we examine the transfer hell arising by intrinsically-typed syntax encodings of System F and System Fω. Specifically, we consider the problems arising with type-renaming-indexed renamings and type-substitution-indexed substitutions. Our solution is to strengthen definitional equality rather than to add more lemmas. We make substitution compute canonically by internalizing the equational theory of substitutions (the σ-calculus) as definitional equality using Agda's rewriting mechanism. As a consequence, extensionally equal substitutions normalize to the same form and transports disappear from routine proofs. This approach is inspired by the tactics of Autosubst [10–12], but internalized by rewriting [4]. We build on a standard functional representation of renamings and substitutions [2], prove the lemmas underlying the equational laws of the sigma calculus, and then inject these equations into Agda's rewrite engine, i.e., make the sigma calculus equations definitional (for type substitutions). The technical challenge is to ensure that the added equations remain terminating and confluent together with the proof assistant's built-in definitional equality.

We first exercise this program for plain System F in Section 2, significantly simplifying the transfer-ridden approach used by Saffrich et al. [9], and then apply the same techniques to System Fω in Section 3. In the latter, the main difference to previous work [3] lies in our intrinsic treatment of type conversion, which leads to significant simplifications. In particular, in our developments, transport lemmas disappear, subject reduction holds by construction, and type conversion in System Fω is handled intrinsically.

In summary, intrinsically typed syntax fails for polymorphic calculi because substitutions lack canonical equality; making them compute canonically restores the original promise of intrinsic typing.

Contributions

This functional pearl demonstrates how to mechanize polymorphic calculi using intrinsically typed syntax without pervasive transport reasoning.

- We identify the fundamental cause of "transfer hell" in intrinsically typed encodings of System F and System Fω: renamings and substitutions indexed by other substitutions produce equal types that are not definitionally equal.
- We show how to eliminate these transports by installing the equational laws of the σ-calculus for substitutions as definitional equality using Agda's rewriting mechanism, taking care to make the equations compatible with the built-in definitional equality.
- We demonstrate that the resulting rewrite system is compatible with Agda's definitional equality (in particular, terminating and confluent) for the functional representation of substitutions and verify that the resulting rewrite system is terminating and confluent with respect to the proof assistant's definitional equality.
- We mechanize parts of the metatheory of System F and extend it to System Fω, obtaining substantially simpler proofs and an intrinsic treatment of type conversion for the latter.
- From this experience we extract practical guidelines for designing intrinsically typed syntax in dependently typed proof assistants.

```
- renamings
_→ᴿ_ : Nat → Nat → Set
n₁ →ᴿ n₂ = Fin n₁ → Fin n₂

opaque
  - weakening
  wkᴿ : n →ᴿ suc n
  wkᴿ = suc

  - identity renaming
  idᴿ : n →ᴿ n
  idᴿ α = α

  - extend with new variable
  _·ᴿ_ : Fin n₂ → n₁ →ᴿ n₂ → suc n₁ →ᴿ n₂
  (α ·ᴿ ζ) zero = α
  (_ ·ᴿ ζ) (suc α) = ζ α

  - apply renaming to variable
  _&ᴿ_ : Fin n₁ → n₁ →ᴿ n₂ → Fin n₂
  α &ᴿ ζ = ζ α

  - composition
  _∘_ : n₁ →ᴿ n₂ → n₂ →ᴿ n₃ → n₁ →ᴿ n₃
  (ζ₁ ∘ ζ₂) α = ζ₂ (ζ₁ α)

- lifting
_↑ᴿ : n₁ →ᴿ n₂ → suc n₁ →ᴿ suc n₂
_↑ᴿ ζ = zero ·ᴿ (ζ ∘ wkᴿ)

- apply renaming to type
_⋯ᴿ_ : Type n₁ → n₁ →ᴿ n₂ → Type n₂
(‘ α)       ⋯ᴿ ζ = ‘ (α &ᴿ ζ)
(∀α T)      ⋯ᴿ ζ = ∀α (T ⋯ᴿ (ζ ↑ᴿ))
(T₁ ⇒ T₂) ⋯ᴿ ζ = (T₁ ⋯ᴿ ζ) ⇒ (T₂ ⋯ᴿ ζ)
```

```
- substitutions
_→ˢ_ : Nat → Nat → Set
n₁ →ˢ n₂ = Fin n₁ → Type n₂

opaque
  - lift renaming to substitution
  ⟨_⟩ : n₁ →ᴿ n₂ → n₁ →ˢ n₂
  ⟨ ζ ⟩ α = ‘ (α &ᴿ ζ)

  - extend with new type
  _·ˢ_ : Type n₂ → n₁ →ˢ n₂ → suc n₁ →ˢ n₂
  (T ·ˢ η) zero = T
  (T ·ˢ η) (suc α) = η α

  - apply substitution to variable
  _&ˢ_ : Fin n₁ → n₁ →ˢ n₂ → Type n₂
  α &ˢ η = η α

  - lifting
  _↑ˢ : n₁ →ˢ n₂ → suc n₁ →ˢ suc n₂
  _↑ˢ η = (‘ zero) ·ˢ λ α → (η α) ⋯ᴿ wkᴿ

- apply substitution to type
_⋯ˢ_ : Type n₁ → n₁ →ˢ n₂ → Type n₂
(‘ α)       ⋯ˢ η = α &ˢ η
(∀α T)      ⋯ˢ η = ∀α (T ⋯ˢ (η ↑ˢ))
(T₁ ⇒ T₂) ⋯ˢ η = (T₁ ⋯ˢ η) ⇒ (T₂ ⋯ˢ η)

opaque
  - composition
  _⨾_ : n₁ →ˢ n₂ → n₂ →ˢ n₃ → n₁ →ˢ n₃
  (η₁ ⨾ η₂) α = (η₁ α) ⋯ˢ η₂
```

Fig. 2. Renamings and substitutions on types

2 System F

2.1 Types

The syntax of System F types uses the standard de Bruijn encoding of variables by natural numbers. The index n stands for the number of type variables in scope. Variables are drawn from the type Fin n, i.e., the set $\{0, \ldots, n-1\}$. The $\forall\alpha$-binder introduces a new type variable, and $\Rightarrow$ is the infix function type constructor.

```
data Type (n : Nat) : Set where
  ‘_    : Fin n → Type n
```

```
∀α : Type (suc n) → Type n
_⇒_ : Type n → Type n → Type n
```

Figure 2 contains the first step in defining type renamings (left column) and substitutions (right column). A type renaming ζ maps variables from one scope n_1 to another n_2. We write the type of type renamings as $n_1 \rightarrow^R n_2$. The definitions comprise (in order) weakening, the identity renaming, extending a renaming, applying a renaming to a variable, composing two renamings, lifting a renaming (to move under a binder), and applying a renaming to a type.

A this point, the astute reader may wonder about the use of opaque in the definitions. The function symbols defined in an opaque block are usable from outside, but their computational behavior is hidden. Thus, the only normal definition is $\uparrow^R$ for lifting a renaming. This selective hiding is essential for our approach because we want to establish new definitional equalities (i.e., the laws of the sigma calculus with first-class renamigns) on top of the thus defined symbols. These new equalities interfere in a bad way with the standard definitional equalities, thus we hide the latter. To prove these equalities, we open the opaque definitions locally and proceed as usual.

For substitutions (right column of Figure 2) we proceed analogously. A type substitution η maps variables from scope n_1 to types defined over scope n_2. We write the type of type substitutions as $n_1 \rightarrow^S n_2$. The definitions (in order) comprise lifting a renaming to a substitution, extending a substitution, applying a substitution to variable, lifting a substitution (to move under a binder), applying a substitution to a type, and composing two substitutions.

Figure 3 gives the equational laws of the sigma calculus with first-class renamings [10]. All of these equalities are supported by proofs using the definitions from Figure 2, but the proofs are not included because they are standard.

The first group gives computation rules for renamings, in particular, how to apply the different forms of renamings to a variable. The second group presents interaction laws for renamings. Most of them deal with composition.

The third group presents computation rules for substitutions starting with the application of a substitution to a variable. There is one equation for each possible form of a substitution: extended substitution, lifted renaming, and composition of substitutions. The final equation in this group characterizes lifting in terms of extension, weakening, and composition.

The fourth group contains laws about composition of substitutions, which are mostly analogous to the corresponding laws for renamings.

The fifth group presents laws that govern the composition of different kinds of term traversal. All four combinations of applying renamings and substitutions to a type are covered.

The final group contains covers laws that mediate between substitutions and renamings. The extraction of renamings from substitutions in general requires a dedicated solving strategy [12] and we have omitted some additionally required coincidence laws from the figure.

Orienting these equations from left to right results in a confluent and terminating rewrite system that we install in Agda using its rewrite pragma [4]. From now on, all equations in Figure 3 are definitional!

As a sanity check, we show that lifting a renaming as well as the action of a renaming satisfy the functor laws. All of these lemmas hold definitionally, some correspond directly equations from the sigma calculus in Figure 3. The latter are marked with $*$.[2]

```
lift*-id : (id^R {n} ↑^R) ≡ id^R
lift*-id = refl

lift*-comp : (ζ′ ∘ ζ) ↑^R ≡ (ζ′ ↑^R) ∘ (ζ ↑^R)
```

[2]The naming of the lemmas is taken from Chapman et al. [3] to enable direct comparisons.

$$
\begin{array}{lll}
\multicolumn{3}{l}{\text{-- computing renamings}} \\
\text{`beta-ext-zero} & : \mathsf{zero}\ \&^R\ (\alpha \cdot^R \zeta) & \equiv \alpha \\
\text{`beta-ext-suc} & : \mathsf{suc}\ \alpha\ \&^R\ (\alpha' \cdot^R \zeta) & \equiv \alpha\ \&^R\ \zeta \\
\text{`beta-id} & : \alpha\ \&^R\ \mathsf{id}^R & \equiv \alpha \\
\text{`beta-wk} & : \alpha\ \&^R\ \mathsf{wk}^R & \equiv \mathsf{suc}\ \alpha \\
\text{`beta-comp} & : \alpha\ \&^R\ (\zeta_1 \circ \zeta_2) & \equiv (\alpha\ \&^R\ \zeta_1)\ \&^R\ \zeta_2 \\
\multicolumn{3}{l}{\text{-- interaction between renamings}} \\
\text{`associativity} & : (\zeta_1 \circ \zeta_2) \circ \zeta_3 & \equiv \zeta_1 \circ (\zeta_2 \circ \zeta_3) \\
\text{`distributivity} & : (\alpha \cdot^R \zeta_1) \circ \zeta_2 & \equiv (\alpha\ \&^R\ \zeta_2) \cdot^R (\zeta_1 \circ \zeta_2) \\
\text{`interact} & : \mathsf{wk}^R \circ (\alpha \cdot^R \zeta) & \equiv \zeta \\
\text{`comp-id}_r & : \zeta \circ \mathsf{id}^R & \equiv \zeta \\
\text{`comp-id}_l & : \mathsf{id}^R \circ \zeta & \equiv \zeta \\
\text{`}\eta\text{-id} & : \mathsf{zero}\ \{n\} \cdot^R \mathsf{wk}^R & \equiv \mathsf{id}^R \\
\text{`}\eta\text{-law} & : (\mathsf{zero}\ \&^R\ \zeta) \cdot^R (\mathsf{wk}^R \circ \zeta) & \equiv \zeta
\end{array}
$$

$$
\begin{array}{lll}
\multicolumn{3}{l}{\text{-- computing substitutions}} \\
\text{beta-ext-zero} & : \mathsf{zero}\ \&^S\ (T \cdot^S \eta) & \equiv T \\
\text{beta-ext-suc} & : \mathsf{suc}\ \alpha\ \&^S\ (T \cdot^S \eta) & \equiv \alpha\ \&^S\ \eta \\
\text{beta-rename} & : \alpha\ \&^S\ \langle\ \zeta\ \rangle & \equiv \text{`}\ (\alpha\ \&^R\ \zeta) \\
\text{beta-comp} & : \alpha\ \&^S\ (\eta_1 \fatsemi \eta_2) & \equiv (\alpha\ \&^S\ \eta_1) \cdots^S \eta_2 \\
\text{beta-lift} & : \eta \uparrow^S & \equiv (\text{`}\ \mathsf{zero}) \cdot^S (\eta \fatsemi \langle\ \mathsf{wk}^R\ \rangle) \\
\multicolumn{3}{l}{\text{-- interaction between substitutions}} \\
\text{associativity} & : (\eta_1 \fatsemi \eta_2) \fatsemi \eta_3 & \equiv \eta_1 \fatsemi (\eta_2 \fatsemi \eta_3) \\
\text{distributivity} & : (T \cdot^S \eta_1) \fatsemi \eta_2 & \equiv (T \cdots^S \eta_2) \cdot^S (\eta_1 \fatsemi \eta_2) \\
\text{interact} & : \langle\ \mathsf{wk}^R\ \rangle \fatsemi (T \cdot^S \eta) & \equiv \eta \\
\text{comp-id}_r & : \eta \fatsemi \langle\ \mathsf{id}^R\ \rangle & \equiv \eta \\
\text{comp-id}_l & : \langle\ \mathsf{id}^R\ \rangle \fatsemi \eta & \equiv \eta \\
\eta\text{-id} & : (\text{`}\ \mathsf{zero}\ \{n\}) \cdot^S \langle\ \mathsf{wk}^R\ \rangle & \equiv \langle\ \mathsf{id}^R\ \rangle \\
\eta\text{-law} & : (\mathsf{zero}\ \&^S\ \eta) \cdot^S (\langle\ \mathsf{wk}^R\ \rangle \fatsemi \eta) & \equiv \eta
\end{array}
$$

$$
\begin{array}{lll}
\multicolumn{3}{l}{\text{-- composing renamings and substitutions}} \\
\text{identity}_r & : T \cdots^R \mathsf{id}^R & \equiv T \\
\text{compose}^{RS} & : (T \cdots^R \zeta_1) \cdots^S \eta_2 & \equiv T \cdots^S (\langle\ \zeta_1\ \rangle \fatsemi \eta_2) \\
\text{compose}^{RR} & : (T \cdots^R \zeta_1) \cdots^R \zeta_2 & \equiv T \cdots^R (\zeta_1 \circ \zeta_2) \\
\text{compose}^{SR} & : (T \cdots^S \eta_1) \cdots^R \zeta_2 & \equiv T \cdots^S (\eta_1 \fatsemi \langle\ \zeta_2\ \rangle) \\
\text{compose}^{SS} & : (T \cdots^S \eta_1) \cdots^S \eta_2 & \equiv T \cdots^S (\eta_1 \fatsemi \eta_2) \\
\multicolumn{3}{l}{\text{-- transforming substitutions to renamings}} \\
\text{coincidence} & : T \cdots^S \langle\ \zeta\ \rangle & \equiv T \cdots^R \zeta \\
\text{coincidence-comp} & : \langle\ \zeta_1\ \rangle \fatsemi \langle\ \zeta_2\ \rangle & \equiv \langle\ \zeta_1 \circ \zeta_2\ \rangle
\end{array}
$$

Fig. 3. Equational laws of the sigma calculus with first class renamings

```
weaken : Type n → Type (suc n)
weaken T = T ⋯ᴿ wkᴿ

_[_]* : Type (suc n) → Type n → Type n
T [ T′ ]* = T ⋯ˢ (T′ ·ˢ idˢ)

data Ctx : Nat → Set where
  ∅    : Ctx zero
  _▷_  : Ctx n → Type n → Ctx n
  _▷*  : Ctx n → Ctx (suc n)

data _∋_ : Ctx n → Type n → Set where
  zero : (Γ ▷ T) ∋ T
  suc  : Γ ∋ T → (Γ ▷ T′) ∋ T
  suc* : Γ ∋ T → (Γ ▷*) ∋ (weaken T)

data Expr : Ctx n → Type n → Set where
  ‘_   : Γ ∋ T →
         Expr Γ T
  λx   : Expr (Γ ▷ T₁) T₂ →
         Expr Γ (T₁ ⇒ T₂)
  _·_  : Expr Γ (T₁ ⇒ T₂) →
         Expr Γ T₁ →
         Expr Γ T₂
  Λα   : Expr (Γ ▷*) T →
         Expr Γ (∀α T)
  _·*_ : Expr Γ (∀α T) →
         (T′ : Type n) →
         Expr Γ (T [ T′ ]*)
```

Fig. 4. Type contexts, variables, and expressions

```
lift*-comp = refl

ren*-id : T ⋯ᴿ idᴿ ≡ T
ren*-id = refl    - *

ren*-comp : T ⋯ᴿ (ζ′ ∘ ζ) ≡ (T ⋯ᴿ ζ′) ⋯ᴿ ζ
ren*-comp = refl - *
```

We also show analogous results for substitution.

```
lifts*-id : (idˢ {n} ↑ˢ) ≡ idˢ
lifts*-id = refl

lifts*-comp : (η′ ⨾ η) ↑ˢ ≡ (η′ ↑ˢ) ⨾ (η ↑ˢ)
lifts*-comp = refl

sub*-id : T ⋯ˢ idˢ ≡ T
sub*-id = refl

sub*-var : (‘ α) ⋯ˢ η ≡ α &ˢ η
sub*-var = refl    - *

sub*-comp : T ⋯ˢ (η ⨾ η′) ≡ (T ⋯ˢ η) ⋯ˢ η′
sub*-comp = refl - *
```

2.2 Expressions

Working towards intrinsically-typed expressions, Figure 4 defines type weakening and single substitution along with type contexts, expression variables, and the syntax of expressions. Type contexts and expression variables follow the work of Chapman et al. [3]. Type contexts Ctx keep track of expression variables and type variables. Correspondingly, variables are implemented as typed de Bruijn indices with an extra case suc* to skip over type variables in the context. Skipping requires weakening of the type as it is used under one more type binding.

Expressions are indexed by a context and a type. Each expression constructor corresponds to a typing rule in System F. The rule for type application relies on single substitution for types.

The definition of a small-step operational semantics requires expression renamings and substitution to implement beta reduction for value abstractions and type abstractions. As before, we aim to define a call-by-name semantics for concision.

2.3 Expression Renaming and Substitution

For expressions, we represent renamings and substitutions directly by suitable functions, rather than axiomatic as with type renamings and substitutions. Hence, we cannot automate the simplification of expression substitutions; nevertheless, we define the primitives analogously.

2.3.1 Renaming. Expression renamings are indexed by type renamings because type variables in expression may have to be renamed at the same time as expression variables. This complication makes the types of the operations quite involved, although the actual operations are not that complicated.

```
_|_⇒ᴿ_ : n₁ →ᴿ n₂ → Ctx n₁ → Ctx n₂ → Set
ζ | Γ₁ ⇒ᴿ Γ₂ = ∀ T → Γ₁ ∋ T → Γ₂ ∋ (T ⋯ᴿ ζ)
```

Defining the identity renaming is straightforward due to the presence of rewriting for type substitutions.

```
Idᴿ : idᴿ | Γ ⇒ᴿ Γ
Idᴿ _ x = x
```

Weakening of a renaming by a value binding.

```
Wkᴿ : ∀ T → idᴿ | Γ ⇒ᴿ (Γ ▷ T)
Wkᴿ _ _ = suc
```

Weakening of a renaming by a type binding.

```
wkᴿ* : wkᴿ | Γ ⇒ᴿ (Γ ▷*)
wkᴿ* _ x = suc* x
```

Extension of a renaming.

```
_|_·ᴿ_ : ∀ ζ → Γ₂ ∋ (T ⋯ᴿ ζ) → ζ | Γ₁ ⇒ᴿ Γ₂ → ζ | (Γ₁ ▷ T) ⇒ᴿ Γ₂
(_ | x ·ᴿ ρ) _ zero    = x
(_ | _ ·ᴿ ρ) _ (suc x) = ρ _ x
```

Lifting of a renaming over a value binding.

```
_|_⇑ᴿ_ : ∀ ζ → ζ | Γ₁ ⇒ᴿ Γ₂ → ∀ T → ζ | (Γ₁ ▷ T) ⇒ᴿ (Γ₂ ▷ (T ⋯ᴿ ζ))
(ζ | ρ ⇑ᴿ _) = ζ | zero ·ᴿ (ζ , idᴿ | ρ ⊙ (Wkᴿ _))
```

Lifting of a renaming over a type binding.

```
_|_↑ᴿ* : ∀ ζ → ζ | Γ₁ ⇒ᴿ Γ₂ → (ζ ↑ᴿ) | (Γ₁ ▷*) ⇒ᴿ (Γ₂ ▷*)
(_ | ρ ↑ᴿ*) _ (suc* x) = suc* (ρ _ x)
```

Inductive definition of a renaming on expressions. The last case of the definition requires rewriting to type check.

```
_|_⋯ᴿ_ : (ζ : n₁ →ᴿ n₂) → Expr Γ₁ T → ζ | Γ₁ ⇒ᴿ Γ₂ → Expr Γ₂ (T ⋯ᴿ ζ)
_ | (` x)      ⋯ᴿ ρ = ` (ρ _ x)
_ | (λx e)     ⋯ᴿ ρ = λx (_ | e ⋯ᴿ (ρ ⇑ᴿ _))
```

```
_ | (Λα e)   ⋯ᴿ ρ = Λα (_ | e ⋯ᴿ (↑ᴿ* ρ))
_ | (e₁ · e₂) ⋯ᴿ ρ = (_ | e₁ ⋯ᴿ ρ) · (_ | e₂ ⋯ᴿ ρ)
ζ | (e ·* T′) ⋯ᴿ ρ = (ζ | e ⋯ᴿ ρ) ·* (T′ ⋯ᴿ ζ)
```

2.3.2 Substitution. Expression substitutions are indexed by type substitutions.

```
_|_⇒ˢ_ : n₁ →ˢ n₂ → Ctx n₁ → Ctx n₂ → Set
η | Γ₁ ⇒ˢ Γ₂ = ∀ T → Γ₁ ∋ T → Expr Γ₂ (T ⋯ˢ η)
```

The identity substitution for expressions is straightforward to define. But this simplicity is due to the installed rewriting of type substitutions, which avoids an explicit application of a transport lemma using comp-id$_r$.

```
Idˢ : idˢ | Γ ⇒ˢ Γ
Idˢ _ = ‘_
```

Extending a substitution is standard.

```
_|_·ˢ_ : ∀ η → Expr Γ₂ (T ⋯ˢ η) → η | Γ₁ ⇒ˢ Γ₂ → η | (Γ₁ ▷ T) ⇒ˢ Γ₂
(_ | e ·ˢ σ) _ zero    = e
(_ | e ·ˢ σ) _ (suc x) = σ _ x
```

Due to the structure of contexts, we need two lifting lemmas, one for value bindings and another for type bindings. The former requires a transport lemma, which is applied by rewriting.

```
_|_⇑ˢ_ : ∀ η → η | Γ₁ ⇒ˢ Γ₂ → ∀ T → η | (Γ₁ ▷ T) ⇒ˢ (Γ₂ ▷ (T ⋯ˢ η))
η | σ ⇑ˢ T = η | (‘ zero) ·ˢ λ _ x → idᴿ | (σ _ x) ⋯ᴿ (Wkᴿ _)
```

Lifting a substitution over a type binding is straightforward.

```
_|_↑ˢ* : ∀ η → η | Γ₁ ⇒ˢ Γ₂ → (η ↑ˢ) | (Γ₁ ▷*) ⇒ˢ (Γ₂ ▷*)
(η | σ ↑ˢ*) _ (suc* x) = _ | (σ _ x) ⋯ᴿ wkᴿ*
```

These operations are sufficient to define the application of a substitution to an expression.

```
_|_⋯ˢ_ : (η : n₁ →ˢ n₂) → Expr Γ₁ T → η | Γ₁ ⇒ˢ Γ₂ → Expr Γ₂ (T ⋯ˢ η)
η | (‘ x)     ⋯ˢ σ = σ _ x
η | (λx e)    ⋯ˢ σ = λx (η | e ⋯ˢ (η | σ ⇑ˢ _))
η | (Λα e)    ⋯ˢ σ = Λα ((η ↑ˢ) | e ⋯ˢ (η | σ ↑ˢ*))
η | (e · e₁)  ⋯ˢ σ = (η | e ⋯ˢ σ) · (η | e₁ ⋯ˢ σ)
η | (e ·* T′) ⋯ˢ σ = (η | e ⋯ˢ σ) ·* (T′ ⋯ˢ η)
```

2.4 Semantics

We are now in position to define a small-step operational semantics. The definition of single substitution, needed for beta reduction, is analogous to the one on types: we build the substitution by extending the identity substitution with the argument. In addition, a type substitution into expression is needed for beta reduction of type applications. This substitution is also implemented by an expression substitution indexed by a type substitution that performs the actual change.

```
_[_] : Expr (Γ ▷ T′) T → Expr Γ T′ → Expr Γ T
e [ e′ ] = idˢ | e ⋯ˢ (idˢ | e′ ·ˢ Idˢ)

_[*_*] : Expr (Γ ▷*) T → (T′ : Type n) → Expr Γ (T [ T′ ]*)
e [* T′ *] = (T′ ·ˢ idˢ) | e ⋯ˢ (idˢ | T′ ·ˢ* Idˢ)
```

```
data Value : Expr Γ T → Set where
  λx : (e : Expr (Γ ▷ T₁) T₂) → Value (λx e)
  Λα : Value e → Value (Λα e)

data Progress : Expr Γ T → Set where
  done : (v : Value e) → Progress e
  step : (e⟶e′ : e ⟶ e′) → Progress e

NoVar : Ctx n → Set
NoVar ∅ = ⊤
NoVar (Γ ▷ x) = ⊥
NoVar (Γ ▷*) = NoVar Γ

noVar : NoVar Γ → ¬ (Γ ∋ T)
noVar nv (suc* x) = noVar nv x

progress : NoVar Γ → (e : Expr Γ T) → Progress e
progress nv (` x) = ⊥-elim (noVar nv x)
progress nv (λx e) = done (λx e)
progress nv (e · e₁)
  with progress nv e
... | done (λx e₂) = step β-λ
... | step e⟶e′ = step (ξ-· e⟶e′)
progress nv (Λα e)
  with progress nv e
... | done v = done (Λα v)
... | step e⟶e′ = step (ξ-Λ e⟶e′)
progress nv (e ·* T′)
  with progress nv e
... | done (Λα v) = step β-Λ
... | step e⟶e′ = step (ξ-·* e⟶e′)
```

Fig. 5. Progress for System F

With substitution in place, the definition of call-by-name small-step reduction is straightforward. As usual, such a definition of reduction for intrinsically-typed syntax implies subject reduction by construction.

```
data _⟶_ : Expr Γ T → Expr Γ T → Set where
  β-λ : (λx e₁ · e₂) ⟶ (e₁ [ e₂ ])
  β-Λ : (Λα e ·* T′) ⟶ (e [* T′ *])
  ξ-·  : e₁ ⟶ e₁′ → (e₁ · e₂) ⟶ (e₁′ · e₂)
  ξ-·* : e ⟶ e′ → (e ·* T) ⟶ (e′ ·* T)
  ξ-Λ : e ⟶ e′ → (Λα e) ⟶ (Λα e′)
```

To obtain type safety, it remains to prove progress. Figure 5 presents the complete proof of progress for call-by-name System F. A value is either a lambda or a big lambda where the body is already a value. An expression fulfills progress if it is either a value (done) or it can make a step (step). Due to the definition of value for big lambdas, we establish progress for type contexts that may contain type bindings, but no value bindings. These contexts are characterized by function NoVar. The actual proof of progress is by induction on the structure of expressions. The overall structure of this proof closely follows the corresponding proof by [3]. Their proof is more complicated because their semantics is call-by-value and their language supports isorecursive types.

2.5 Laws for Renamings and Substitutions

Proving type safety for System F *requires* exercising the substitution theory at the *type level*. Without installing its laws as definitional equalities, many definitions become more cumbersome because transport lemmas have to be inserted to equalize the types of (Agda) expressions.

While proving type safety does *not* require exercising same theory at the *expression level*, it is still good practice to verify functorial and monadic properties of expression substitutions. Moreover, if we were to move on to constructing and proving properties of logical relations, then these properties would be needed [9].

As an example, we examine the composition of two expression substitutions. Its definition involves two type substitutions η_1 and η_2 that index the corresponding expression substitutions σ_1 and σ_2. The composed expression substitution is indexed by the composition of η_1 and η_2. Type checking this definition already exercises the laws for composition of type substitutions.

```
_,_|_⨾_ : ∀ η₁ η₂ → η₁ | Γ₁ ⇒ˢ Γ₂ → η₂ | Γ₂ ⇒ˢ Γ₃ → (η₁ ⨾ η₂) | Γ₁ ⇒ˢ Γ₃
(_ , _ | σ₁ ⨾ σ₂) _ x = _ | (σ₁ _ x) ⋯ˢ σ₂
```

Here is the statement of functoriality of expression substitution: applying two substitutions one after the other to an expression is the same as applying their composition. This statement is a very prominent example of transfer hell: the types of the left and right side of the equality are different and would require a transfer without our rewriting machinery in place.

```
Composeˢˢ : ∀ (e : Expr Γ₁ T) (η₁ : n₁ →ˢ n₂) (η₂ : n₂ →ˢ n₃) (σ₁ : η₁ | Γ₁ ⇒ˢ Γ₂) (σ₂ : η₂ | Γ₂ ⇒ˢ Γ₃) →
  η₂ | (η₁ | e ⋯ˢ σ₁) ⋯ˢ σ₂ ≡ (η₁ ⨾ η₂) | e ⋯ˢ (η₁ , η₂ | σ₁ ⨾ σ₂)
```

We show the proof of this property just do demonstrate that type adjustments for type substitutions can be ignored and we can concentrate on the gist of the proof, rather than being overwhelmed by transfers. The remaining fly in the ointment is the explicit passing of the type substitutions η_1 and η_2. We chose to make them explicit; while Agda can infer them in many places, there are equally many places where inference is not possible.

```
Composeˢˢ (` x)     η₁ η₂ σ₁ σ₂ = refl
Composeˢˢ (λx e)    η₁ η₂ σ₁ σ₂ = cong λx (begin
                                    _ ≡⟨ Composeˢˢ e η₁ η₂ (η₁ | σ₁ ⇑ˢ _) (η₂ | σ₂ ⇑ˢ _) ⟩
                                    _ ≡⟨ cong ((η₁ ⨾ η₂) | e ⋯ˢ_) (Lift-Dist-Compˢˢ σ₁ σ₂) ⟩
                                    _ ∎)
Composeˢˢ (Λα e)    η₁ η₂ σ₁ σ₂ = cong Λα (begin
                                    _ ≡⟨ Composeˢˢ e (η₁ ↑ˢ) (η₂ ↑ˢ) (η₁ | σ₁ ↑ˢ*) (η₂ | σ₂ ↑ˢ*) ⟩
                                    _ ≡⟨ cong (((η₁ ⨾ η₂) ↑ˢ) | e ⋯ˢ_) (lift*-dist-Compˢˢ η₁ η₂ σ₁ σ₂) ⟩
                                    _ ∎)
Composeˢˢ (e₁ · e₂) η₁ η₂ σ₁ σ₂ = cong₂ _·_ (Composeˢˢ e₁ η₁ η₂ σ₁ σ₂) (Composeˢˢ e₂ η₁ η₂ σ₁ σ₂)
Composeˢˢ (e ·* T′) η₁ η₂ σ₁ σ₂ = cong (_·* (T′ ⋯ˢ (η₁ ⨾ η₂))) (Composeˢˢ e η₁ η₂ σ₁ σ₂)
```

This proof relies on further properties, in particular Lift-Dist-Comp^{SS} (lifting a composition of substitutions is equal to composing their liftings) and lift*-dist-Comp^{SS} (its counterpart for lifting over type bindings), where the statement and proof would require explicit transfers. Essentially, these properties form a "food chain" that stops at the composition properties of renamings with renamings and liftings, all with similar issues. The supplemental material gives the full story.

2.6 Comparison to Proof in Transfer Hell

We compare this proof to the one without the additional definitional equalities for type substitutions, provided in the supplement for the aforementioned logical relation construction [9]. Here we only state the *type* of the proof and the induction case for application, which is among the short ones.

```
(e : Expr Δ₁ Γ₁ T) (σ₁ : ESub σ₁* Γ₁ Γ₂) → (σ₂ : ESub σ₂* Γ₂ Γ₃) →
    let sub = subst (Expr Δ₃ Γ₃) (fusion-Tsub-Tsub T σ₁* σ₂*) in
    sub (Esub σ₂* σ₂ (Esub σ₁* σ₁ e)) ≡ Esub (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂) e
```

The mapping from their to our syntax (in order) is

- $\Delta \mapsto n$

- ESub ↦ _|_ ⋯ ^{S}_
- σ^* ↦ η
- fusion-Tsub-Tsub ↦ compose^{SS}
- _∘^{SS} ↦ _⨾_
- _»^{SS}_ ↦ _|_⨾_

The authors use heterogeneous equality to proof the lemma, thereby avoiding some of the troubles arising from transfer hell.

```
R.begin
  Esub σ₂* σ₂ (Esub σ₁* σ₁ (e₁ · e₂))
R.≅⟨ refl ⟩
  Esub σ₂* σ₂ (Esub σ₁* σ₁ e₁) · Esub σ₂* σ₂ (Esub σ₁* σ₁ e₂)
R.≅⟨ Hcong₄ {C = λ ■₁ ■₂ → Expr Δ₃ Γ₃ (■₂ ⇒ ■₁)} {D = λ _ ■₂ _ → Expr Δ₃ Γ₃ ■₂} (λ _ _ → _·_)
            (H.≡-to-≅ (fusion-Tsub-Tsub T σ₁* σ₂*)) (H.≡-to-≅ (fusion-Tsub-Tsub T₁ σ₁* σ₂*))
            (fusion-Esub-Esub' e₁ σ₁ σ₂) (fusion-Esub-Esub' e₂ σ₁ σ₂) ⟩
  Esub (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂) e₁ · Esub (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂) e₂
R.≅⟨ refl ⟩
  Esub (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂) (e₁ · e₂)
R.□
```

The full proof can be found in Appendix A

2.7 Examples

As an example of System F expressions, we present some definitions for Church numerals. We start with their type. For readability, we define aliases for type variables like ‘α for ‘ Z, ‘β for ‘ (S Z), and analogously for expression variables and binders.

```
ℕᶜ : Type ∅ ∗
ℕᶜ = ∀α ((‘α ⇒ ‘α) ⇒ (‘α ⇒ ‘α))
```

We give the definitions for some constants and the successor operation.

```
zeroᶜ : Expr ∅ ℕᶜ
zeroᶜ = Λα (λg (λx ‘x))

oneᶜ : Expr ∅ ℕᶜ
oneᶜ = Λα (λg (λx (‘g · ‘x)))

succᶜ : Expr ∅ (ℕᶜ ⇒ ℕᶜ)
succᶜ = λx (Λα (λg (λx (‘g · ((((‘ (S (S (S* Z)))) • ‘α) · ‘g) · ‘x)))))

twoᶜ : Expr ∅ ℕᶜ
twoᶜ = succᶜ · (succᶜ · zeroᶜ)
```

3 System Fω

System Fω extends System F with higher-kinded types, that is, abstraction over type functions. This calculus is the basis of Haskell's type system where type functions are limited to partial application of type constructors. Abstraction of higher-kinded types is key to defining type classes for functors, monads, etc. An extension of Fω with isorecursive types is the basis for Plutus, the language of the Cardano blockchain [3].

3.1 Definitions

We only give a glimpse of the definitions of the syntax of types and expressions, which are taken from [3] and hence generalize the definitions for System F from Section 2. We do not reiterate the development of renamings and substitutions, as it transfers mutatis mutandis: the typings get more involved, but the equational laws remain exactly the same.

For System Fω, type variables have more structure as in System F, as there are arrow kinds in addition to plain types. We use I, J, K to range over types.

```
data Kind : Set where
  ∗ : Kind
  _⇒_ : Kind → Kind → Kind
```

Consequently, kind contexts are not mere numbers, but lists of kinds that grow to the right. Kind contexts are ranged over by Φ, Ψ, Θ.

```
data Ctx* : Set where
  ∅ : Ctx*
  _▷*_ : Ctx* → Kind → Ctx*
```

Type variables have the same structure as plain variables in the simply-typed lambda calculus (see Figure 1).

```
data _∋*_ : Ctx* → Kind → Set where
  Z : (Φ ▷* K) ∋* K
  S : Φ ∋* K → (Φ ▷* J) ∋* K
```

The definition of the type syntax also mimicks the expressions of the simply-typed lambda calculus, but with an important twist, which deviates from the prior work.

```
data Type Φ : Kind → Set where
  ‘_    : Φ ∋* K → Type Φ K
  λα : Type (Φ ▷* J) K → Type Φ (J ⇒ K)
  - _$_  : Type Φ (J ⇒ K) → Type Φ J → Type Φ K
  ∀α : Type (Φ ▷* J) ∗ → Type Φ ∗
  _⇒_ : Type Φ ∗ → Type Φ ∗ → Type Φ ∗
postulate
  _$_  : Type Φ (J ⇒ K) → Type Φ J → Type Φ K
```

As System Fω enjoys full beta reduction at the type level, we do not define application (with symbol \$) as a data constructor. Instead, we introduce it as a symbol and use rewriting to extend functions on the type syntax to include application.

As an example, we show the definition of applying a renaming to an expression. We elide the preceding definitions of lifting, extension, and the type substitution machinery because they are identical to the ones for System F in Section 2.1, save for the more elaborate type signatures.

```
_⋯ᴿ_ : Type Φ K → Φ →ᴿ Ψ → Type Ψ K
(‘ x) ⋯ᴿ ρ = ‘ ρ x
(λα T) ⋯ᴿ ρ = λα (T ⋯ᴿ ↑ᴿ ρ)
- (T₁ $ T₂) ⋯ᴿ ρ = (T₁ ⋯ᴿ ρ) $ (T₂ ⋯ᴿ ρ)
(∀α T) ⋯ᴿ ρ = ∀α (T ⋯ᴿ ↑ᴿ ρ)
(T₁ ⇒ T₂) ⋯ᴿ ρ = (T₁ ⋯ᴿ ρ) ⇒ (T₂ ⋯ᴿ ρ)
```

```
postulate
  traverseᴿ-$ : (T₁ $ T₂) ⋯ᴿ ρ ≡ (T₁ ⋯ᴿ ρ) $ (T₂ ⋯ᴿ ρ)
```

On top of the rewriting theory for type substitutions, we install type-level beta reduction as a rewrite rule, using single type substitution on the right side:

```
postulate
  β≡* : (λα T₁) $ T₂ ≡ T₁ [ T₂ ]*

{-# REWRITE β≡* #-}
```

The combined theory is still confluent and terminating [1, 5].

Surprisingly, type-level beta reduction is not required to define the syntax of expressions, expression renamings and substitutions, defining the operational semantics, as well as the proof of type safety. Thanks to definitional beta conversion at the type level, the expression syntax does *not* require an explicit type conversion rule as it was the case in prior work [3]. Hence, the proof presented in Figure 5 for System F carries over, mutatis mutandis (i.e., adapting the type signatures).

In the next section, we develop some examples that genuinely rely on type conversion.

3.2 Examples

Chapman et al. [3] consider several examples to demonstrate their encoding of System F$\omega\mu$. Their first group of examples covers Church numerals in System F. These are covered in Section 2.7. Their second group of example covers recursive types. We have no counterpart for them. Unfortunately, they do not provide examples that exercise the full power of System Fω, namely higher-kinded types! We fill this gap in the following.

3.2.1 Church Numerals. First, we encode (simply-typed) Church numerals *in the type language*. As beta reduction on the type level is definitional in our development, normalization of Church numerals is performed by Agda's type checker.

The kind of type level Church numerals is as expected.

```
ℕᵏ : Kind
ℕᵏ = (∗ ⇒ ∗) ⇒ (∗ ⇒ ∗)
```

We define zero and successor for Church numerals in the usual way. For readability, we define aliases like λβ and so on for the type level lambda λα. Analogously, we define aliases for type variables like ‘α for ‘ Z, ‘β for ‘ (S Z), and so on.

```
zeroᵏ : Type ∅ ℕᵏ
zeroᵏ = λβ (λα ‘α)

succᵏ : Type ∅ (ℕᵏ ⇒ ℕᵏ)
succᵏ = λγ (λβ (λα (‘β $ ((‘γ $ ‘β) $ ‘α))))
```

We construct the Church numeral for 2 by applying the successor twice to zero. The proof that this term is equal to is normal form is definitional.

```
twoᵏ : Type ∅ ℕᵏ
twoᵏ = succᵏ $ (succᵏ $ zeroᵏ)

_ : twoᵏ ≡ λβ (λα (‘β $ (‘β $ ‘α)))
_ = refl
```

Finally, we define addition and construct 2 by adding $1 + 1$. The corresponding equality proof puts a significant load on the type checker and takes more than a minute to complete on the authors' machine.

```
one^k : Type ∅ ℕ^k
one^k = λβ (λα (‘β $ ‘α))

add^k : Type ∅ (ℕ^k ⇒ (ℕ^k ⇒ ℕ^k))
add^k = λα (λα (λα (λα ((((‘ S (S (S Z))) $ (‘ S Z)) $ ((((‘ S (S Z)) $ (‘ S Z)) $ (‘ Z))))))

_ : two^k ≡ (add^k $ one^k) $ one^k
_ = refl
```

3.2.2 Arrows. The final example is inspired by the work on arrows [6]. This work uses type classes to abstract over different instantiations of the function arrow. For clarity, we start with an Agda version of the function that we wish to implement in System Fω.

The function abst is a glorified identity function. It abstracts over a binary type constructor κ, two types β and α, then it takes an abstract application function *app* and applies it to an abstract function f and a suitable argument x. Function inst instantiates this function. It acts like an eliminator for abst. The function use puts both together.

```
ty : Set₁
ty = (κ : Set → Set → Set) (β α : Set) (app : κ β α → β → α) (f : κ β α) (x : β) → α

abst : ty
abst = λ κ β α → λ app f x → app f x

inst : ∀ α → ty → α → α
inst = λ α g x → g (λ β α → (β → α)) α α (λ f x → f x) (λ z → z) x

use : ∀ α → α → α
use = λ α x → inst α abst x
```

We start by defining the type in our encoding.

```
ty-enc : Type Φ ∗
ty-enc = ∀κ (∀β (∀α ((((‘κ $ ‘β) $ ‘α) ⇒ (‘β ⇒ ‘α))
                  ⇒ (((‘κ $ ‘β) $ ‘α) ⇒ (‘β ⇒ ‘α)))))
```

Next, we encode the defining expression equivalent to abst.

```
abst-enc : Expr Γ ty-enc
abst-enc = Λκ (Λβ (Λα (λf (λy (λx ((‘f · ‘y) · ‘x))))))
```

To use this function, we encode functions inst and use.

```
inst-enc : Expr Γ (∀α (ty-enc ⇒ (‘α ⇒ ‘α)))
inst-enc = Λα (λg (λx ((((((‘g • (λβ (λα (‘β ⇒ ‘α)))) • ‘α) • ‘α)
                        · (λy (λx (‘y · ‘x))))
                        · (λx ‘x))
                        · ‘x)))

use-enc : Expr ∅ (∀α (‘α ⇒ ‘α))
use-enc = Λα (λx (((inst-enc • ‘α) · abst-enc) · ‘x))
```

The definitions of the encodings require beta conversion at the type level. These definitions are not accepted without the rewriting of beta reduction $\beta\equiv^*$.

4 Conclusion

Intrinsically typed syntax promises elegant mechanizations by making well-typed programs representable only as well-typed data, but for polymorphic calculi this promise is often undermined by pervasive transport reasoning. Our experience with System F and System Fω shows that the difficulty does not stem from binding structure or indexing itself, but from a mismatch between substitution and definitional equality: substitutions that are extensionally equal fail to compute to a canonical form. By internalizing the equational laws of the σ-calculus as definitional equality, substitutions become canonical and routine transports disappear from proofs. The resulting developments recover the simplicity seen in the simply-typed case while scaling to higher-order polymorphism and intrinsic type conversion. More broadly, this pearl suggests a practical guideline for mechanizing type systems in dependently typed proof assistants: when intrinsically typed encodings become proof-heavy, a promising strategy is often to strengthen definitional equality rather than introduce additional lemmas or automation.

References

[1] M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Levy. 1989. Explicit substitutions. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) *(POPL '90)*. Association for Computing Machinery, New York, NY, USA, 31–46. doi:10.1145/96709.96712

[2] Nick Benton, Chung-Kil Hur, Andrew Kennedy, and Conor McBride. 2012. Strongly Typed Term Representations in Coq. *J. Autom. Reason.* 49, 2 (2012), 141–159. doi:10.1007/S10817-011-9219-0

[3] James Chapman, Roman Kireev, Chad Nester, and Philip Wadler. 2019. System F in Agda, for Fun and Profit. In *Mathematics of Program Construction: 13th International Conference, MPC 2019, Porto, Portugal, October 7–9, 2019, Proceedings* (Porto, Portugal). Springer-Verlag, Berlin, Heidelberg, 255–297. doi:10.1007/978-3-030-33636-3_10

[4] Jesper Cockx, Nicolas Tabareau, and Théo Winterhalter. 2021. The Taming of the Rew: A Type Theory with Computational Assumptions. *Proc. ACM Program. Lang.* 5, POPL, Article 60 (Jan. 2021), 29 pages. doi:10.1145/3434341

[5] Pierre-Louis Curien, Thérèse Hardin, and Jean-Jacques Lévy. 1996. Confluence properties of weak and strong calculi of explicit substitutions. *J. ACM* 43, 2 (March 1996), 362–397. doi:10.1145/226643.226675

[6] John Hughes. 2000. Generalising Monads to Arrows. *Sci. Comput. Program.* 37, 1-3 (2000), 67–111. doi:10.1016/S0167-6423(99)00023-4

[7] Wen Kokke, Jeremy G. Siek, and Philip Wadler. 2020. Programming Language Foundations in Agda. *Sci. Comput. Program.* 194 (2020), 102440. doi:10.1016/J.SCICO.2020.102440

[8] Conor McBride. 2000. Elimination with a Motive. In *Types for Proofs and Programs, International Workshop, TYPES 2000, Durham, UK, December 8-12, 2000, Selected Papers (Lecture Notes in Computer Science, Vol. 2277)*, Paul Callaghan, Zhaohui Luo, James McKinna, and Robert Pollack (Eds.). Springer, 197–216. doi:10.1007/3-540-45842-5_13

[9] Hannes Saffrich, Peter Thiemann, and Marius Weidner. 2024. Intrinsically Typed Syntax, a Logical Relation, and the Scourge of the Transfer Lemma. In *Proceedings of the 9th ACM SIGPLAN International Workshop on Type-Driven Development* (Milan, Italy) *(TyDe 2024)*. Association for Computing Machinery, New York, NY, USA, 2–15. doi:10.1145/3678000.3678201

[10] Steven Schäfer, Tobias Tebbi, and Gert Smolka. 2015. Autosubst: Reasoning with de Bruijn Terms and Parallel Substitutions. In *Interactive Theorem Proving*, Christian Urban and Xingyuan Zhang (Eds.). Springer International Publishing, Cham, 359–374. https://www.ps.uni-saarland.de/Publications/documents/SchaeferEtAl_2015_Autosubst_-Reasoning.pdf

[11] Kathrin Stark. 2020. *Mechanising Syntax with Binders in Coq.* Ph. D. Dissertation. Saarland University, Saarbrücken, Germany. https://www.ps.uni-saarland.de/Publications/documents/Stark_2020_Mechanising.pdf Dissertation zur Erlangung des Grades des Doktors der Ingenieurwissenschaften (Dr.-Ing.).

[12] Kathrin Stark, Steven Schäfer, and Jonas Kaiser. 2019. Autosubst 2: reasoning with multi-sorted de Bruijn terms and vector substitutions *(CPP 2019)*. Association for Computing Machinery, New York, NY, USA, 166–180. doi:10.1145/3293880.3294101

A Proof for Compose Monad Law in Transfer Hell

```
fusion-Esub-Esub {Δ₁} {Δ₂} {Δ₃} {l} {σ*} {ρ*} {Γ₁} {Γ₂} {Γ₃} {T} e σ ρ =
  let F₁ = (Expr _ Γ₃) ; E₁ = (fusion-Tsub-Tsub T σ* ρ*) ; sub₁ = subst F₁ E₁ in
  H.≅-to-≡ (
    R.begin
      sub₁ (Esub ρ* ρ (Esub σ* σ e))
    R.≅⟨ H.≡-subst-removable F₁ E₁ _ ⟩
      Esub ρ* ρ (Esub σ* σ e)
    R.≅⟨ fusion-Esub-Esub' e σ ρ ⟩
      Esub (σ* ∘ₛₛ ρ*) (σ »SS ρ) e
    R.□
  )
fusion-Esub-Esub' :
    {σ₁* : TSub Δ₁ Δ₂} {σ₂* : TSub Δ₂ Δ₃}
  → {Γ₁ : TEnv Δ₁}{Γ₂ : TEnv Δ₂}{Γ₃ : TEnv Δ₃}
  → {T : Type Δ₁ l}
  → (e : Expr Δ₁ Γ₁ T)
  → (σ₁ : ESub σ₁* Γ₁ Γ₂) (σ₂ : ESub σ₂* Γ₂ Γ₃)
  → Esub σ₂* σ₂ (Esub σ₁* σ₁ e) ≅ Esub (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂) e
fusion-Esub-Esub' {Δ₁} {Δ₂} {Δ₃} {.zero} {σ₂*} {σ₁*} {Γ₁} {Γ₂} {Γ₃} {.`ℕ} (# n) σ₁ σ₂ =
  refl
fusion-Esub-Esub' {Δ₁} {Δ₂} {Δ₃} {.zero} {σ₂*} {σ₁*} {Γ₁} {Γ₂} {Γ₃} {.`ℕ} (`suc e) σ₂ σ₁ =
  R.begin
    Esub σ₁* σ₁ (Esub σ₂* σ₂ (`suc e))
  R.≅⟨ refl ⟩
    `suc (Esub σ₁* σ₁ (Esub σ₂* σ₂ e))
  R.≅⟨ H.cong `suc (fusion-Esub-Esub' e σ₂ σ₁) ⟩
    `suc (Esub (σ₂* ∘ₛₛ σ₁*) (σ₂ »SS σ₁) e)
  R.≅⟨ refl ⟩
    Esub (σ₂* ∘ₛₛ σ₁*) (σ₂ »SS σ₁) (`suc e)
  R.□
fusion-Esub-Esub' {Δ₁} {Δ₂} {Δ₃} {l} {σ₁*} {σ₂*} {Γ₁} {Γ₂} {Γ₃} {T} (` x) σ₁ σ₂ =
  let F₁ = (Expr Δ₃ Γ₃) ; E₁ = (fusion-Tsub-Tsub T σ₁* σ₂*) ; sub₁ = subst F₁ E₁ in
  R.begin
    Esub σ₂* σ₂ (Esub σ₁* σ₁ (` x))
  R.≅⟨ refl ⟩
    Esub σ₂* σ₂ (σ₁ l T x)
  R.≅⟨ H.sym (H.≡-subst-removable F₁ E₁ _) ⟩
    sub₁ (Esub σ₂* σ₂ (σ₁ l T x))
  R.≅⟨ refl ⟩
    Esub (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂) (` x)
  R.□
fusion-Esub-Esub' {Δ₁} {Δ₂} {Δ₃} {_} {σ₁*} {σ₂*} {Γ₁} {Γ₂} {Γ₃} {T₁ ⇒ T₂} (ƛ e) σ₁ σ₂ =
  R.begin
    ƛ Esub σ₂* (Elifts σ₂* σ₂) (Esub σ₁* (Elifts σ₁* σ₁) e)
  R.≅⟨ Hcong₃ {C = λ ■₁ ■₂ → Expr Δ₃ (■₁ ◃ Γ₃) ■₂} (λ _ _ ■ → ƛ ■)
              (H.≡-to-≅ (fusion-Tsub-Tsub T₁ σ₁* σ₂*))
              (H.≡-to-≅ (fusion-Tsub-Tsub T₂ σ₁* σ₂*))
              (fusion-Esub-Esub-lift e σ₁ σ₂) ⟩
    ƛ (Esub (σ₁* ∘ₛₛ σ₂*) (Elifts (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂)) e)
  R.≅⟨ refl ⟩
    Esub (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂) (ƛ e)
```

```
  R.□
fusion-Esub-Esub' {Δ₁} {Δ₂} {Δ₃} {l} {σ₁*} {σ₂*} {Γ₁} {Γ₂} {Γ₃} {T} (_·_ {T = T₁} {T′ = T₂} e₁ e₂) σ₁ σ₂ =
  R.begin
    Esub σ₂* σ₂ (Esub σ₁* σ₁ (e₁ · e₂))
  R.≅⟨ refl ⟩
    Esub σ₂* σ₂ (Esub σ₁* σ₁ e₁) · Esub σ₂* σ₂ (Esub σ₁* σ₁ e₂)
  R.≅⟨ Hcong₄ {C = λ ■₁ ■₂ → Expr Δ₃ Γ₃ (■₂ ⇒ ■₁)} {D = λ _ ■₂ _ → Expr Δ₃ Γ₃ ■₂} (λ _ _ → _·_)
                (H.≡-to-≅ (fusion-Tsub-Tsub T σ₁* σ₂*)) (H.≡-to-≅ (fusion-Tsub-Tsub T₁ σ₁* σ₂*))
                (fusion-Esub-Esub' e₁ σ₁ σ₂) (fusion-Esub-Esub' e₂ σ₁ σ₂) ⟩
    Esub (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂) e₁ · Esub (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂) e₂
  R.≅⟨ refl ⟩
    Esub (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂) (e₁ · e₂)
  R.□
fusion-Esub-Esub' {Δ₁} {Δ₂} {Δ₃} {_} {σ₁*} {σ₂*} {Γ₁} {Γ₂} {Γ₃} {`∀α l , T} (Λ l ⇒ e) σ₁ σ₂ =
  R.begin
    Esub σ₂* σ₂ (Esub σ₁* σ₁ (Λ l ⇒ e))
  R.≅⟨ refl ⟩
    Λ l ⇒ Esub (Tliftₛ σ₂* l) (Eliftₛ-l σ₂* σ₂) (Esub (Tliftₛ σ₁* l) (Eliftₛ-l σ₁* σ₁) e)
  R.≅⟨ H.cong₂ {B = Expr (l :: Δ₃) (l ◃* Γ₃)} (λ _ → Λ l ⇒_)
                (H.≡-to-≅ (fusion-Tsub-Tsub T (Tliftₛ σ₁* _) (Tliftₛ σ₂* _)))
                (fusion-Esub-Esub' e (Eliftₛ-l σ₁* σ₁) (Eliftₛ-l σ₂* σ₂)) ⟩
    Λ l ⇒ Esub (Tliftₛ σ₁* l ∘ₛₛ Tliftₛ σ₂* l) (Eliftₛ-l σ₁* σ₁ »SS Eliftₛ-l σ₂* σ₂) e
  R.≅⟨ H.cong₂ {B = Expr (l :: Δ₃) (l ◃* Γ₃)} (λ _ → Λ l ⇒_)
                (H.≡-to-≅ (cong (λ σ₁ → Tsub σ₁ T) (sym (sub↑-dist-∘ₛₛ _ σ₁* σ₂*))))
                (fusion-Esub-Esub-lift-l e σ₁ σ₂) ⟩
    Λ l ⇒ Esub (Tliftₛ (σ₁* ∘ₛₛ σ₂*) l) (Eliftₛ-l (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂)) e
  R.≅⟨ refl ⟩
    Esub (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂) (Λ l ⇒ e)
  R.□
fusion-Esub-Esub' {Δ₁} {Δ₂} {Δ₃} {l} {σ₁*} {σ₂*} {Γ₁} {Γ₂} {Γ₃} {_} (_·_ {T = T} e T′) σ₁ σ₂ =
  let
    F₂ = Expr Δ₂ Γ₂ ; E₂ = sym (swap-Tsub-[] σ₁* T T′) ; sub₂ = subst F₂ E₂
    F₃ = Expr Δ₃ Γ₃ ; E₃ = sym (swap-Tsub-[] (σ₁* ∘ₛₛ σ₂*) T T′) ; sub₃ = subst F₃ E₃
    F₅ = Expr Δ₃ Γ₃ ; E₅ = sym (swap-Tsub-[] σ₂* (Tsub (Tliftₛ σ₁* _) T) (Tsub σ₁* T′)) ; sub₅ = subst F₅ E₅
  in
  R.begin
    Esub σ₂* σ₂ (Esub σ₁* σ₁ (e · T′))
  R.≅⟨ refl ⟩
    Esub σ₂* σ₂ (sub₂ (Esub σ₁* σ₁ e · Tsub σ₁* T′))
  R.≅⟨ H.cong₂ {B = Expr Δ₂ Γ₂} (λ _ ■ → Esub σ₂* σ₂ ■) (H.≡-to-≅ (sym E₂))
                                (H.≡-subst-removable F₂ E₂ _) ⟩
    Esub σ₂* σ₂ (Esub σ₁* σ₁ e · Tsub σ₁* T′)
  R.≅⟨ refl ⟩
    sub₅ (Esub σ₂* σ₂ (Esub σ₁* σ₁ e) · Tsub σ₂* (Tsub σ₁* T′))
  R.≅⟨ H.≡-subst-removable F₅ E₅ _ ⟩
    Esub σ₂* σ₂ (Esub σ₁* σ₁ e) · Tsub σ₂* (Tsub σ₁* T′)
  R.≅⟨ Hcong₃ {B = λ ■ → Expr Δ₃ Γ₃ (`∀α _ , ■)} {C = λ _ _ → Type Δ₃ _ } (λ _ ■₁ ■₂ → ■₁ · ■₂)
        (H.≡-to-≅ (fusion-Tsub-Tsub-lift T σ₁* σ₂*)) (fusion-Esub-Esub' e σ₁ σ₂)
        (H.≡-to-≅ (fusion-Tsub-Tsub T′ σ₁* σ₂*)) ⟩
    Esub (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂) e · Tsub (σ₁* ∘ₛₛ σ₂*) T′
  R.≅⟨ H.sym (H.≡-subst-removable F₃ E₃ _) ⟩
    sub₃ (Esub (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂) e · (Tsub (σ₁* ∘ₛₛ σ₂*) T′))
```

```
R.≅⟨ refl ⟩
  Esub (σ₁* ∘ₛₛ σ₂*) (σ₁ »SS σ₂) (e · T′)
R.□
```